

Botton-up

```
In [1]: def fibonacci_iterativo_v1(numero):  
        f1 = 0  
        f2 = 1  
        tmp = 0  
        for i in range(1, numero-1):  
            tmp = f1 + f2  
            f1 = f2  
            f2 = tmp  
        return f2
```

```
In [2]: fibonacci_iterativo_v1(6)
```

Out[2]: 5

```
In [3]: def fibonacci_iterativo_v2(numero):  
        f1=0  
        f2=1  
        for i in range(1, numero-1):  
            f1,f2=f2, f1+f2  
        return f2
```

```
In [4]: fibonacci_iterativo_v2(6)
```

Out[4]: 5

```
In [5]: def fibonacci_bottom_up(numero):  
        f_parciales = [0, 1, 1]  
        while len(f_parciales) < numero:  
            f_parciales.append(f_parciales[-1] + f_parciales[-2])  
            print(f_parciales)  
        return f_parciales[numero-1]
```

```
In [6]: fibonacci_bottom_up(5)
```

[0, 1, 1, 2]

[0, 1, 1, 2, 3]

Out[6]: 3

Top-down

```
In [7]: memoria = {1:0, 2:1, 3:1}
```

```
In [9]: def fibonacci_top_down(numero):  
        if numero in memoria:  
            return memoria[numero]
```

```
f = fibonacci_iterativo_v2(numero-1) + fibonacci_iterativo_v2(numero-2)
memoria[numero] = f
return memoria[numero]
```

```
In [10]: fibonacci_top_down(12)
```

```
Out[10]: 89
```

```
In [11]: memoria
```

```
Out[11]: {1: 0, 2: 1, 3: 1, 12: 89}
```

```
In [12]: fibonacci_top_down(8)
```

```
Out[12]: 13
```

```
In [13]: memoria
```

```
Out[13]: {1: 0, 2: 1, 3: 1, 12: 89, 8: 13}
```

```
In [15]: import pickle
```

```
archivo = open('memoria.p', 'wb')
pickle.dump(memoria, archivo)
archivo.close()
```

```
In [16]: archivo = open('memoria.p', 'rb')
memoria_de_archivo = pickle.load(archivo)
archivo.close()
```

```
In [17]: memoria
```

```
Out[17]: {1: 0, 2: 1, 3: 1, 12: 89, 8: 13}
```

```
In [18]: memoria_de_archivo
```

```
Out[18]: {1: 0, 2: 1, 3: 1, 12: 89, 8: 13}
```

Quicksort

```
In [19]: def quicksort(lista):
          quicksort_aux(lista, 0, len(lista)-1)

          def quicksort_aux(lista, inicio, fin):
              if inicio < fin:
                  pivote = particion(lista, inicio, fin)

                  quicksort_aux(lista, inicio, pivote-1)
                  quicksort_aux(lista, pivote+1, fin)
```

```
In [21]: def particion(lista, inicio, fin):
    pivote = lista[inicio]
    print("Valor del pivote {}".format(pivote))
    izquierda = inicio+1
    derecha = fin
    print("Indice izquierdo {}".format(izquierda))
    print("Indice derecho {}".format(derecha))

    bandera = False
    while not bandera:
        while izquierda <= derecha and lista[izquierda] <= pivote:
            izquierda = izquierda + 1
        while lista[derecha] >= pivote and derecha >= izquierda:
            derecha = derecha - 1
        if derecha < izquierda:
            bandera = True
        else:
            temp = lista[izquierda]
            lista[izquierda] = lista[derecha]
            lista[derecha] = temp

    print(lista)

    temp=lista[inicio]
    lista[inicio]=lista[derecha]
    lista[derecha]=temp
    return derecha
```

```
In [22]: lista = [21, 10, 0, 11, 9, 24, 20, 14, 1]
    print("lista desordenada {}".format(lista))

    quicksort(lista)
    print("lista ordenada {}".format(lista))
```

```
lista desordenada [21, 10, 0, 11, 9, 24, 20, 14, 1]
Valor del pivote 21
Indice izquierdo 1
Indice derecho 8
[21, 10, 0, 11, 9, 1, 20, 14, 24]
Valor del pivote 14
Indice izquierdo 1
Indice derecho 6
[14, 10, 0, 11, 9, 1, 20, 21, 24]
Valor del pivote 1
Indice izquierdo 1
Indice derecho 4
[1, 0, 10, 11, 9, 14, 20, 21, 24]
Valor del pivote 10
Indice izquierdo 3
Indice derecho 4
[0, 1, 10, 9, 11, 14, 20, 21, 24]
lista ordenada [0, 1, 9, 10, 11, 14, 20, 21, 24]
```